

PGCP-ITISS | February 2026

DevSecOps Automated Security Pipeline

Industry-grade team project guide

Python	Linux	Docker	Kubernetes	AWS EKS	Jenkins
OWASP ZAP	Trivy	Wazuh	ELK Stack	Snort	Prometheus

Covers all 8 ITISS modules • 3-person team • 12-week roadmap

Designed to be interview-ready and industry-demanded

Table of contents

1. Project overview & why this project
2. Full architecture — all 6 layers
3. Complete tools & technology stack
4. 12-week phase-by-phase roadmap
5. Team role split (3 members)
6. Syllabus module mapping
7. Interview talking points
8. Phase deep-dives
9. Project folder structure
10. Evaluation & grading alignment

1. Project overview & why this project

The **DevSecOps Automated Security Pipeline** is a production-grade, end-to-end project that integrates security at every stage of the software delivery lifecycle. Instead of treating security as a final checkpoint, this pipeline shifts security left — catching vulnerabilities at the source code stage, container build stage, and runtime stage, all automatically.

This project is intentionally designed to mirror what companies like Infosys, Wipro, TCS, HCL, and security-focused startups are actively hiring for in 2025-26. It demonstrates competence across networking, OS administration, programming, cloud, DevOps, and security — all in one cohesive deliverable.

Why this is industry-demanded

Business problems this project solves
Insecure code reaching production — solved by SAST (Bandit) and secrets scanning (Gitleaks) at commit time
Vulnerable container images deployed to cloud — solved by Trivy image scanning in the CI gate
No visibility into runtime threats — solved by Wazuh HIDS + Snort NIDS + ELK log correlation
Manual compliance reporting — solved by Python dashboard auto-generating NIST/ISO 27001 reports
Hardcoded secrets in code or K8s manifests — solved by AWS Secrets Manager + K8s RBAC policies

2. Full architecture — all 6 layers

The pipeline is organized into six distinct layers. Each layer has a clear security responsibility and maps directly to topics covered in the PGCP-ITISS syllabus.

Layer 1 — Dev

GitHub repository with branch protection rules and mandatory code review

Pre-commit hooks: Bandit (Python SAST) runs before every git commit

Gitleaks: scans for accidentally committed API keys, passwords, tokens

Developer workstation: Ubuntu Linux with Python 3.x virtual environment

Layer 2 — CI/CD Pipeline

Jenkins server (on Linux VM or AWS EC2) triggered on every GitHub push

Docker builds the application image using a hardened Dockerfile

Trivy scans the built image for CVEs — pipeline fails if CRITICAL CVEs found

Image is signed using cosign before being pushed to DockerHub

Jenkinsfile written as declarative pipeline with parallel security stages

Layer 3 — Security Gate

Python script reads Trivy JSON output and calculates a composite risk score

OWASP ZAP performs DAST (Dynamic Application Security Testing) against test environment

Policy engine: blocks deployment if risk score exceeds configurable threshold

All scan results stored in MySQL database for historical trending

Layer 4 — Deploy: Kubernetes + AWS

AWS EKS cluster with separate namespaces: dev, staging, production

Kubernetes RBAC: each team member has least-privilege service accounts

NetworkPolicies: pods can only communicate with explicitly allowed services

AWS Secrets Manager: all credentials injected as K8s secrets at runtime

VPC with public subnet (ALB) and private subnets (app + database pods)

Layer 5 — Observe & Respond

Prometheus scrapes metrics from all pods via node-exporter and kube-state-metrics

Grafana dashboards: cluster health, pod restart counts, network traffic anomalies

ELK Stack (Elasticsearch + Logstash + Kibana): centralised log aggregation

Wazuh agent on all nodes: HIDS detecting file integrity changes, rootkits
Snort/Suricata: NIDS watching for port scans, exploit attempts, C2 traffic
Alerting: PagerDuty / email when critical rules fire

Layer 6 — Compliance & Audit

Python Flask web dashboard reads scan history from MySQL
Auto-maps findings to NIST CSF controls (Identify, Protect, Detect, Respond, Recover)
Generates ISO 27001 Annex A control compliance summary
Exportable PDF report for management / project submission
Remediation tracker: open issues, assigned owner, due date, status

3. Complete tools & technology stack

Tool	Purpose	Layer
GitHub	Source control, branch protection, webhooks to Jenkins	Dev
Bandit	Python static analysis — finds insecure code patterns	Dev
Gitleaks	Detect secrets/credentials accidentally committed	Dev
Jenkins	CI/CD orchestration, Jenkinsfile declarative pipelines	CI/CD
Docker	Container build, hardened images, multi-stage builds	CI/CD
Trivy	Container image CVE scanner, IaC misconfiguration scanner	CI/CD
cosign	Container image signing for supply chain integrity	CI/CD
OWASP ZAP	DAST — automated web application vulnerability scanning	Security
Metasploit	Controlled pen-testing in isolated lab environment	Security
Python (scripting)	Risk scoring engine, compliance dashboard, automation glue	Security
MySQL	Vulnerability history, scan results, remediation tracker	Security
AWS EKS	Managed Kubernetes cluster, node groups, IAM integration	Cloud
AWS VPC	Network isolation, public/private subnets, security groups	Cloud
AWS Secrets Mgr	Centralised secret store, K8s ExternalSecrets integration	Cloud
Kubernetes	Container orchestration, RBAC, NetworkPolicy, namespaces	Cloud
Ansible	Configuration management, K8s node setup automation	Cloud
Terraform	Infrastructure as Code — VPC, EKS cluster provisioning	Cloud
Prometheus	Metrics scraping, alerting rules, PromQL queries	Monitor
Grafana	Dashboards for cluster, app, and security metrics	Monitor
ELK Stack	Log aggregation (Logstash), search (Elastic), visualise (Kibana)	Monitor
Wazuh	HIDS — file integrity monitoring, rootkit detection, compliance	Monitor
Snort/Suricata	NIDS — real-time network intrusion detection	Monitor
pfSense	Firewall, IDS/IPS integration, VPN (OpenVPN)	Monitor
OpenSSL / XCA	PKI — Root CA, Sub CA, TLS certificates for HTTPS	PKI
Flask	Compliance dashboard web application	Compliance

4. 12-week phase-by-phase roadmap

Phase	Timeline	Focus	Key deliverable
Phase 1	Weeks 1-2	Foundation & Dev layer setup	GitHub repo + pre-commit hooks working, Bandit and Gitleaks blocking bad commits
Phase 2	Weeks 3-4	CI/CD pipeline	Jenkins pipeline: build Docker image, Trivy scan, push to DockerHub on pass
Phase 3	Weeks 5-6	Security gate (Python)	Automated risk-scoring script, OWASP ZAP DAST, block/pass decision engine
Phase 4	Weeks 7-8	Kubernetes + AWS deploy	EKS cluster live, RBAC + NetworkPolicies applied, Secrets Manager integrated
Phase 5	Weeks 9-10	Monitoring + SIEM	Prometheus/Grafana live, ELK ingesting logs, Wazuh + Snort alerting on threats
Phase 6	Weeks 11-12	Compliance dashboard + documentation	Flask dashboard with NIST mapping, PDF report generation, project presentation

Weekly checklist approach

Each week should end with a working demo of that week's deliverable. Use GitHub Issues to track tasks. Weekly 30-minute team sync to review blockers.

- Week 1: All 3 members can clone repo, trigger Bandit scan, see results locally
- Week 2: Gitleaks catches a test credential before commit — demo this to each other
- Week 3: Jenkins job triggers on GitHub push, builds Docker image successfully
- Week 4: Trivy fails the pipeline when a known-vulnerable base image is used
- Week 5: Python script outputs a risk score and blocks/passes based on threshold
- Week 6: OWASP ZAP report generated against a running DVWA container
- Week 7: kubectl get pods shows app running in EKS dev namespace
- Week 8: Attempt to access another namespace's secrets — RBAC denies it (demo)
- Week 9: Prometheus graph shows CPU/memory of all pods over time
- Week 10: Wazuh alert fires when a test file is modified on a monitored node
- Week 11: Flask dashboard shows last 10 scan results with NIST control mapping
- Week 12: Full end-to-end demo: code push → scan → deploy → alert → compliance report

5. Team role split (3 members)

Member 1	Member 2	Member 3
DevSecOps Lead	Security Engineer	Cloud & Monitoring
Jenkins CI/CD setup Dockerfile hardening Trivy integration GitHub Actions Kubernetes deployment Helmcharts / manifests	Bandit + Gitleaks setup OWASP ZAP automation Python risk scoring Wazuh configuration Snort rules writing PKI + TLS certificates	AWS EKS provisioning Terraform IaC scripts Prometheus + Grafana ELK stack config Flask compliance dash MySQL schema + reports

Shared responsibilities (all members): Linux server administration, Python scripting, Git workflow, weekly security review meetings, documentation writing, and project presentation preparation.

6. Syllabus module mapping

Every module from the PGCP-ITISS syllabus is represented in this project. The table below shows how each module contributes to a specific layer of the pipeline.

Module	Name	CE	Lab	IA	Total	Used in project
ITISS01	Fundamentals of Computer Networks	40	40	20	100	Networking layer, VPC, ACLs
ITISS02	Concepts of OS & Administration	40	40	20	100	Linux admin, Bash scripts
ITISS03	Programming Concepts	40	40	20	100	Python automation, MySQL
ITISS04	IT Infra Management & DevOps	40	40	20	100	Docker, K8s, Jenkins, AWS
ITISS05	Network Defense & Countermeasures	40	40	20	100	Snort, iptables, SIEM, VPN
ITISS06	Security Concepts	40	40	20	100	OWASP ZAP, Metasploit, SAST
ITISS07	Cyber Forensics + PKI	40	40	20	100	TLS certs, evidence chain
ITISS08	Compliance Audit	40	—	20	—	NIST, ISO 27001 report

Evaluation alignment

Each module uses the same evaluation weightage: Theory exam 40%, Lab exam 40%, Internal Assessment 20%. This project directly strengthens Lab exam performance for ITISS01–ITISS07 and provides real case study material for ITISS08 Compliance Audit theory exam.

7. Interview talking points

Each layer of this project directly answers a frequently asked interview question. Prepare a 2-minute explanation for each point below.

Q: How do you enforce security in a CI/CD pipeline?

Strong answer: We built a Jenkins pipeline where Trivy scans every Docker image before it can be pushed. A Python script reads the JSON output, calculates a CVSS-weighted risk score, and automatically fails the pipeline if the score exceeds our threshold. No human approval needed — security is enforced by code.

Q: How do you handle least privilege in cloud environments?

Strong answer: In our EKS cluster, every application has its own Kubernetes service account bound to an IAM role with only the permissions it needs. We also applied NetworkPolicies so pods can only talk to explicitly whitelisted services. We tested this by attempting cross-namespace secret access and confirmed it was denied.

Q: How would you detect an intrusion in production?

Strong answer: We run Wazuh agents on every node for host-based detection — file integrity monitoring, rootkit detection, and log analysis. Snort handles network-based detection. All alerts feed into the ELK stack where we wrote correlation rules to detect patterns like repeated failed logins followed by privilege escalation attempts.

Q: How do you demonstrate security posture to management?

Strong answer: Our Flask compliance dashboard automatically maps every finding from Trivy and OWASP ZAP to NIST CSF and ISO 27001 Annex A controls. Managers can see a trend graph of vulnerabilities over time, open vs resolved counts, and download a PDF report. No manual spreadsheet work required.

Q: What is shift-left security and how did you implement it?

Strong answer: Shift-left means catching security issues as early as possible — ideally before code is even committed. We used pre-commit hooks with Bandit for Python SAST and Gitleaks for secrets detection. Developers get immediate feedback on their local machine before the code ever reaches the CI server.

Q: How do you manage secrets securely in Kubernetes?

Strong answer: We never hardcode secrets in manifests or environment variables. All credentials are stored in AWS Secrets Manager and injected into pods at runtime using the Kubernetes External Secrets Operator. This means even if someone gets access to our GitHub repo, they find no usable credentials.

8. Phase deep-dives

Phase 1 — Foundation & Dev layer (Weeks 1-2)

Set up your project repository and ensure every code push is automatically screened for security issues before it enters the CI pipeline.

Step-by-step setup

- Create a GitHub organisation repo: devsecops-pipeline. Add all 3 members as collaborators.
- Set branch protection on main: require pull request review + passing status checks.
- Install pre-commit: pip install pre-commit. Create .pre-commit-config.yaml.
- Add Bandit hook: repo: <https://github.com/PyCQA/bandit>, rev: 1.7.8, hooks: id: bandit.
- Add Gitleaks hook: repo: <https://github.com/gitleaks/gitleaks>, rev: v8.18.1.
- Test by writing intentionally insecure Python: subprocess.call(input()) — Bandit should flag it.
- Test Gitleaks by adding a fake AWS key — it must block the commit.
- Document all findings in your project wiki.

Phase 2 — CI/CD Pipeline (Weeks 3-4)

Build the automated pipeline that goes from code push to a scanned, signed, and stored container image — with no human intervention required.

Jenkinsfile stages to implement

```
stage('Checkout') { git branch: 'main', url: 'https://github.com/org/repo' }
stage('Build image') { sh 'docker build -t myapp:${BUILD_NUMBER} .' }
stage('Trivy scan') { sh 'trivy image --exit-code 1 --severity CRITICAL myapp:${BUILD_NUMBER}' }
stage('Sign image') { sh 'cosign sign myapp:${BUILD_NUMBER}' }
stage('Push image') { sh 'docker push registry/myapp:${BUILD_NUMBER}' }
```

Phase 3 — Security Gate (Weeks 5-6)

The Python risk-scoring engine is the heart of your security gate. It transforms raw scanner output into an actionable pass/fail decision.

Python risk scorer — key logic

```
import json, sys
CVSS_WEIGHTS = {'CRITICAL': 10, 'HIGH': 5, 'MEDIUM': 2, 'LOW': 0.5}
THRESHOLD = 30 # configurable per environment
def score_report(trivy_json_path):
    with open(trivy_json_path) as f:
        data = json.load(f)
    score = 0
    for result in data.get('Results', []):
        for vuln in result.get('Vulnerabilities', []):
            score += CVSS_WEIGHTS.get(vuln['Severity'], 0)
    return score
score = score_report('trivy-report.json')
print(f'Risk score: {score}')
sys.exit(1 if score > THRESHOLD else 0)
```

Phase 4 — Kubernetes + AWS (Weeks 7-8)

Deploy to a real cloud environment with security controls that match what enterprises use in production.

Key Kubernetes security manifests to write

- NetworkPolicy: deny all ingress/egress by default, then explicitly allow only needed ports
- RBAC: create ServiceAccount per app, bind to Role with minimal verbs (get, list only)
- PodSecurityContext: runAsNonRoot: true, readOnlyRootFilesystem: true, allowPrivilegeEscalation: false
- ExternalSecret: reference AWS Secrets Manager ARN, sync to K8s Secret automatically
- ResourceQuota: limit CPU and memory per namespace to prevent noisy-neighbour attacks

Phases 5 & 6 — Monitor, SIEM & Compliance (Weeks 9-12)

The final phases complete the observe-and-respond capability and produce the compliance evidence your organisation (or examiner) needs.

Key Wazuh rules to configure

- Rule 550: Integrity checksum changed — fires when a monitored file is modified
- Rule 5503: SSH brute force — fires after 8 failed login attempts in 60 seconds
- Rule 80792: Possible web attack — fires on SQL injection patterns in Apache logs
- Custom rule: Container escape attempt — fires when /proc/self/exe is accessed from a pod

9. Project folder structure

```
devsecops-pipeline/ ■■■ .pre-commit-config.yaml # Bandit + Gitleaks hooks ■■■
Jenkinsfile # Declarative pipeline definition ■■■ Dockerfile # Hardened
multi-stage build ■■■ docker-compose.yml # Local dev environment ■ ■■■ app/ #
Sample Python Flask application ■ ■■■ app.py ■ ■■■ requirements.txt ■ ■■■
tests/ ■ ■■■ security/ ■ ■■■ trivy-scan.sh # Wrapper script for Trivy ■ ■■■
risk_scorer.py # CVE scoring engine ■ ■■■ zap_scan.py # OWASP ZAP automation ■
■■■ policy.yaml # Threshold configuration ■ ■■■ kubernetes/ ■ ■■■ namespace.yaml
■■■ deployment.yaml ■ ■■■ network-policy.yaml ■ ■■■ rbac.yaml ■ ■■■
external-secret.yaml ■ ■■■ terraform/ ■ ■■■ main.tf # EKS cluster + VPC ■ ■■■
variables.tf ■ ■■■ outputs.tf ■ ■■■ monitoring/ ■ ■■■ prometheus/ ■ ■■■
prometheus.yml ■ ■■■ grafana/ ■ ■■■ dashboards/ ■ ■■■ elk/ ■ ■■■
logstash.conf ■ ■■■ compliance/ ■ ■■■ dashboard/ # Flask app ■ ■■■ app.py ■ ■■■
■■■ templates/ ■ ■■■ reports/ # Generated PDF reports ■ ■■■ docs/ ■■■
architecture.md ■■■ runbook.md ■■■ security-findings.md
```

10. Evaluation & grading alignment

The PGCP-ITISS programme evaluates through two centralised examinations. This project strengthens your preparation for both.

Centralised Mid Course Examination (CMCE) — 400 marks

- ITISS01 Networks (100 marks): Your VPC, ACL, VLAN segmentation, and routing knowledge is directly applied in the AWS networking layer. Subnetting lab assignments directly prepare you for the lab exam.
- ITISS02 OS & Administration (100 marks): Linux server hardening, Bash scripting for automation, Active Directory for team access management, and PowerShell for Windows monitoring nodes.
- ITISS03 Programming Concepts (100 marks): Python scripts for risk scoring, MySQL for scan history storage, Flask for the dashboard — all 3 exam topics covered in one project.
- ITISS04 IT Infrastructure Management & DevOps (100 marks): Jenkins, Docker, Kubernetes, Ansible, Terraform, AWS — this is the most heavily weighted DevOps module and this project is essentially a complete practical implementation of the entire syllabus.

Centralised Course End Examination (CCEE) — 300 marks

- ITISS05 Network Defense & Countermeasures (100 marks): pfSense firewall, iptables rules, Snort IDS/IPS, OpenVPN, Wireshark packet analysis, and SIEM deployment all feature prominently in the monitoring layer.
- ITISS06 Security Concepts (100 marks): OWASP Top 10 addressed by ZAP scanning, ethical hacking methodology applied in the pen-test phase, Metasploit for controlled exploitation testing, malware analysis via Wazuh HIDS.
- ITISS07 Cyber Forensics + PKI (100 marks): Chain of custody for incident response, OpenSSL for TLS certificate generation, digital signatures on container images using cosign, evidence collection from ELK logs.
- ITISS08 Compliance Audit (grade-based): The compliance dashboard directly produces NIST CSF and ISO 27001 evidence — this module's theory exam questions on GDPR, PCI-DSS, COBIT, and HIPAA are all addressed through the automated reporting layer.

Project module — ITISS09 (180 lab hours, grade-based)

This project is designed to fulfil the ITISS09 project requirement. It spans all required technical domains, involves team collaboration, produces documented deliverables, and demonstrates industry-relevant skills. The 12-week roadmap aligns with typical project evaluation timelines.

Prepared for PGCP-ITISS February 2026 cohort • CDAC ACTS, Pune • 3-member team project

Best of luck with your project and interviews!